MEMORY & RAG LAB

Giving Your MCP Agent Memory and Grounded Knowledge

THE TASK

You're extending the same company, the same shared GitHub repo, the same database, and the same MCP server from the MCP Server Lab. Your agent can already call scoped tools against real data, but it has two problems: it forgets everything the moment a session ends, and it can't reason over anything that lives outside a tool call, a policy manual, a set of past cases, a body of internal knowledge nobody wants to turn into another dozen MCP tools.

Your job is to find the genuine version of that problem inside your own company and industry, then fix it properly: a real long-term memory system sitting behind your agent, and a real retrieval layer grounding it in documents instead of making things up. This builds directly on top of your existing mcp_server/ and db/. You are not starting a new project.

THE MEMORY & RETRIEVAL CONCERNS

We covered a full memory architecture and a full retrieval architecture, and your system needs to demonstrate all of it, evaluated with real numbers, not just described. **Your team must implement every concern below.** Design your extension so each one has a genuine reason to exist in your system. A consolidation pass over three fake conversations is worse than not having one. Pick a real recurring need for memory and a real body of ungoverned knowledge in your company.

- **Short-term memory and scratchpad:** a rolling message buffer distinct from a scratchpad holding the agent's current plan, sub-goal, and working state, so pruning the transcript never destroys what the agent is actively doing.
- **Context window management, all four strategies:** implement sliding window, observation and tool-output masking, recursive summarization, and zone-based pruning, all four, against the same agent. Build a set of long-context test cases (long tool-heavy transcripts, multi-turn conversations that bury an early decision under later tool noise) and run each strategy against the same test suite. Produce a comparison table scoring each strategy on task accuracy after pruning, tokens consumed, and latency, then pick the one your system actually ships with and justify it against the table, not against intuition.

A cost note worth building into your test design: input tokens are billed and rate-limited far more cheaply than output tokens. That means your best move for building a thorough long-context test suite is to lean on large, realistic input transcripts (bury the decision under thousands of tokens of tool output, don't be shy about it) rather than trying to generate huge amounts of model output, which is where you'll actually burn budget and hit rate limits fastest.

- **Promote-or-drop routing (forget and episodic only):** the decision layer that fires when short-term memory overflows. For each aging item, the router decides to forget it or promote it to episodic memory, with the reasoning behind each decision logged somewhere a grader can see it. This router does not write directly to semantic memory.
- **A consolidation layer for semantic memory:** semantic memory is only ever built through a separate, periodic consolidation pass over the episodic store, never written to directly by the router above. Your consolidation layer has to solve the actual problems semantic facts run into in production: updates when a fact changes, versioning so an old fact isn't silently lost, expiration for facts that go stale, and conflict resolution when two episodes imply contradictory facts. Show a real conflict your consolidation layer resolves, not a hypothetical one.

- **Vector database architecture:** a real vector store, not a list of floats in a Python dict, with an ANN index (HNSW or equivalent), a metadata payload store, and a metadata index that lets you filter before or during similarity search, not just after.
- **Retrieval architectures, three required, one bonus:** implement all three of naive RAG (the baseline chunk, embed, index, retrieve, generate pipeline), hybrid search (vector similarity plus keyword or BM25 in the same query), and agentic RAG (a reasoning loop that decides what to retrieve, retrieves, observes, and decides whether to retrieve again). Graph RAG is a bonus, worth extra credit if it's genuinely applicable to your data (your documents have real entity relationships worth modeling as a graph, not a flat pile of unrelated passages).

Build a set of domain-specific test questions (at least one that should specifically favor each architecture: something naive RAG handles fine, something with exact identifiers that hybrid search should win on, something that needs decomposition or multiple retrieval rounds that only agentic RAG can handle well). Run every architecture against every question and produce a comparison table across accuracy against your test set, token usage per query, and latency per query. Then choose the architecture you'd actually ship, based on your company's real query patterns and the numbers in your table, not based on which one sounds the most advanced.

- **Self-RAG-style verification:** before an answer reaches the user, an explicit check, is the retrieved content actually relevant, is the generated answer actually supported by it, rather than trusting whatever the nearest-neighbor search or hybrid ranker handed back. This applies to both your RAG answers and to memories recalled from the episodic and semantic store.

If your project only demonstrates concerns that would work identically on three lines of throwaway chat, you haven't found the right problem yet. Go back and pick a memory or knowledge need with enough real volume and real staleness to justify all of it honestly.

A WORKED EXAMPLE, READ THIS, THEN DO SOMETHING DIFFERENT WITH YOUR OWN COMPANY

This shows the shape of the exercise on top of the MCP Server Lab example, not the problem to copy. Extend your own company and your own existing repo.

Picture the same "Larkspur Veterinary Clinics" system from the MCP Server Lab. The MCP server already lets staff look up pets, appointments, and prescriptions. Two new problems show up once real usage starts: front-desk staff re-explain the same client's allergy history every visit, and vets keep asking the assistant questions the database was never built to answer, like sedation protocols for senior dogs with a cardiac history, that only live in a forty-page internal clinical policy binder nobody wants to turn into forty new tools.

Problem 1, context management. Larkspur's longest calls are triage calls: an owner describes symptoms over many turns while the agent pulls appointment history, prior visit notes, and prescription records, each a large JSON tool result. The team builds a 40-turn synthetic test transcript where a critical detail (a penicillin reaction mentioned in turn 3) has to survive until the final turn (turn 40, when the vet asks "any allergy concerns before we prescribe?"), buried under 30+ tool calls in between. They run all four strategies against ten variations of this transcript.

Strategy	Allergy detail recalled correctly	Avg. input tokens/run	Avg. output tokens/run	Avg. latency
Sliding window (last 10 turns)	1/10	4,200	180	0.6s
Observation masking (keep last 3 tool outputs)	9/10	6,800	210	0.9s
Recursive summarization (compact	8/10	5,100	640	2.4s

Strategy	Allergy detail recalled correctly	Avg. input tokens/run	Avg. output tokens/run	Avg. latency
every 15 turns)				
Zone-based pruning (4 zones)	9/10	7,400	260	1.3s

The team picks observation masking. It matches Larkspur's actual failure mode (the bloat is tool JSON, not dialogue) at the lowest latency of the two strategies that reliably preserved the critical detail, and it avoids the extra LLM calls recursive summarization needed, which nearly tripled output tokens for a smaller accuracy gain. Zone-based pruning tied on accuracy but cost more latency for no real benefit given their transcript shape. The table, not a guess, is what the README cites as the justification.

Problem 2, retrieval architecture. The clinical policy binder becomes the RAG corpus: 40 pages, chunked and embedded, with metadata for section, species, and last-reviewed date. The team writes 12 test questions across three categories: general clinical questions ("what's the standard fasting window before sedation?"), citation-heavy questions ("what does Protocol 4.2b say about cardiac-risk patients?"), and multi-part questions that need decomposition ("for a 12-year-old dog with a heart murmur needing a dental cleaning, what pre-op screening and sedation adjustments apply?").

Architecture	Accuracy (12 test questions)	Avg. tokens/query	Avg. latency/query
Naive RAG	7/12	1,900	1.1s
Hybrid search (vector + BM25)	10/12	2,100	1.3s
Agentic RAG (multi-hop)	11/12	5,600	4.8s
Graph RAG (bonus, entities: protocol, condition, drug)	11/12	3,300	2.6s

Naive RAG missed nearly every citation-heavy question, since "4.2b" doesn't embed distinctively. Hybrid search fixed that at almost no extra cost. Agentic RAG matched hybrid search on the citation questions and pulled ahead on the multi-part screening question by retrieving twice, but at more than four times the latency and tokens for one additional correct answer out of twelve. Since Larkspur's real call volume is dominated by quick citation and general questions during live calls, where a vet is waiting on the phone, the team ships hybrid search as the default and routes only the multi-part, decomposition-shaped questions to the agentic path, a decision the comparison table makes obvious rather than assumed.

Each concern here earns its place because the stakes are real: a missed allergy or a fabricated protocol answer is the reason the memory and grounding matter, not a feature added because the list said to.

WHAT TO BUILD AND SUBMIT

- An updated **README**: what memory and knowledge problem you found on top of your existing system, why it's real, and how each concern above shows up in your solution.
- A **memory/** folder: your short-term buffer and scratchpad implementation, your episodic and semantic stores, the promote-or-drop routing logic (forget and episodic only), and your separate consolidation layer for semantic memory, including how it resolves a real contradiction.
- A **context_eval/** folder: your implementations of all four context management strategies, your long-context test suite, and the script or notebook that produced your comparison table.
- A **rag/** (or vector_store/) folder: your chunking and embedding pipeline, your vector database setup (index, metadata store, metadata index), and your naive RAG, hybrid search, and agentic RAG implementations (Graph RAG too, if you attempt the bonus).
- A **retrieval_eval/** folder: your domain-specific test question set and the script or notebook that produced your retrieval comparison table.

- Any changes to mcp_server/ or agent/ needed to wire memory and RAG into the existing agent loop. This should visibly reuse the existing server and database, not duplicate them.
- Inside memory/ and rag/, every concern should be easy for a grader to locate without reading the whole file: the routing decision code, the consolidation trigger and its conflict resolution, the vector index config, the Self-RAG-style check, and so on.
- Both comparison tables (context management and retrieval architecture), embedded in the README with the numbers that drove your final choice in each category.
- A short **demo transcript or recording** showing every concern actually firing: an item surviving promote-or-drop into forget or episodic, consolidation resolving a real contradiction, all four context strategies running against your test suite, all three (or four) retrieval architectures answering the same question, and a Self-RAG-style check catching or passing a retrieval.
- Commit history should show real contributions from all team members. Split memory, context evaluation, retrieval evaluation, and integration work so no one owns more than two of the pieces.

RESOURCES

Code-heavy references you should actually open and copy from, not just skim.

- **Self-RAG paper** (arXiv:2310.11511) and the **official implementation** (github.com/AkariAsai/self-rag): the reflection-token mechanism and training data format, worth reading the actual code for the critique step.
- **LangChain RAG tutorial with code** (python.langchain.com/docs/tutorials/rag): a full working chunk, embed, retrieve, generate pipeline you can adapt directly instead of writing one from scratch.
- **LangGraph agentic RAG cookbook** (langchain-ai.github.io/langgraph/tutorials/rag/langgraph_agentic_rag): a working multi-step retrieval loop with grading and query rewriting, the closest reference implementation to what the agentic RAG concern is asking for.
- **rank_bm25** (PyPI): the fastest way to add a real keyword scorer next to your vector search for hybrid search.
- **Chroma quickstart** (docs.trychroma.com/getting-started) for local prototyping, or **Qdrant quickstart** (qdrant.tech/documentation/quickstart)
- **Neo4j GraphRAG Python package** (github.com/neo4j/neo4j-graphrag-python): if you attempt the Graph RAG bonus, this handles entity and relationship extraction plus graph-based retrieval without building a graph pipeline from scratch.
- **Episodic Memory is the Missing Piece for Long-Term LLM Agents** (arXiv:2502.06975) and **Zep: temporal knowledge graph architecture for agent memory** (arXiv:2501.13956): applied architectures for episodic and semantic separation, consolidation, and contradiction handling worth stealing design patterns from.
- **Model Context Protocol, Resources** (modelcontextprotocol.io/specification): revisit this if your RAG corpus overlaps with something you already expose as a resource. Decide deliberately whether it should stay a resource, move into the vector store, or both.

A FEW GUARDRAILS

- Pick a memory or knowledge need where forgetting or hallucinating actually costs something. If nothing bad happens when the agent forgets or makes something up, you haven't found the right problem yet.
- Consolidation must be a genuinely separate, periodic pass over episodic memory, never summarization happening at write time and never something the promote-or-drop router writes to directly.
- Contradictions in semantic memory must be resolved explicitly, versioned, dated, or flagged, never silently overwritten with no trace of the old fact.
- RAG answers must be grounded only in retrieved content. If your demo shows the agent answering confidently with no retrieved chunk behind the claim, that's a failure to show, not something to edit out.

- Don't rebuild your existing database or MCP server from scratch. This lab is graded on genuine extension of existing work, and duplicated effort will read as such.
- Never commit an API key, embedding-provider credential, or database credential. Use a .env file and confirm it's still in .gitignore.
- Keep your long-context and retrieval test suites fixed once you start evaluating. Changing the test cases between runs invalidates your comparison tables.

WORKING AS A TEAM: ISSUES AND MODULAR WORK

Every GitHub Issue is graded on its rationale, not its existence. An issue that says "add semantic memory" earns nothing. One that states the problem, the constraint, and the acceptance criteria does.

Breaking the project into modular parts: split work across memory/, context_eval/, rag/, retrieval_eval/, and the integration touching agent/ and mcp_server/, and again across the concerns above. Each concern gets its own issue with a single clear owner before any code is written.

Writing an issue with real rationale: for example, "front-desk staff re-ask allergy history every visit because nothing persists past the session" rather than "add episodic memory." Name the constraint that makes the concern genuinely necessary in your system, and write acceptance criteria a teammate could check without asking what you meant.

Assigning, linking, and closing issues: one owner per issue, referenced from the pull request that resolves it ("Closes #-"), with a short closing comment explaining what changed. An issue opened and closed with no linked commit or discussion won't count as attributable teamwork.

GRADING RUBRIC

Your team deliverable is scored out of 100 fixed points, plus up to 5 bonus points for Graph RAG. Every row is binary, full points only when its criteria are genuinely met, not partially. Your final individual grade is 70% this team score plus 30% an individual contribution score built from Git history and issue rationale.

Category	Points	What's assessed
Problem framing and suitability	10	Genuine memory and knowledge gap on top of the existing system, originality vs. the worked example, whether it truly needs every concern below.
Extending the existing system	5	The existing mcp_server/ and db/ are visibly reused, not duplicated. The extension reads as growth on the existing repo.
Short-term memory and scratchpad	5	A real rolling buffer plus a scratchpad distinct from the transcript, and pruning that doesn't destroy the scratchpad.
Context window management, all four strategies	15	All four strategies correctly implemented, a real long-context test suite, a complete comparison table (accuracy, tokens, latency), and a final choice justified by that table.
Promote-or-drop routing (forget and episodic)	6	A real decision point on short-term memory overflow with visible reasoning behind forget vs. episodic routing, and no direct writes to semantic memory.
Semantic memory consolidation layer	10	A genuinely periodic pass over episodic memory that demonstrably handles updates, versioning, expiration, and a real conflict.
Vector database architecture	8	Real ANN index, metadata payload store, and a metadata index used for pre- or mid-search filtering, not a bare list of vectors.
Retrieval architectures, three required	15	Naive RAG, hybrid search, and agentic RAG all implemented, a domain-specific test set covering all three, a complete comparison table (accuracy, tokens, latency), and a final choice justified by the company's actual query

Category	Points	What's assessed
		patterns.
Self-RAG-style verification	8	A real post-retrieval and post-generation check for relevance and support, applied to both RAG and memory recall, with a visible consequence when it fails.
Repository usability and safety	5	Clear organization, demo evidence covering every concern, reproducible setup, no exposed secrets.
Teamwork and issue rationale	3	Issues with genuine rationale, single ownership, and traceable resolution through linked pull requests.
Agent and system integration	10	The agent genuinely uses the memory and retrieval systems in its live loop, an end-to-end demo path, and complete run instructions.
Total	**100**	
Bonus: Graph RAG	+5	A genuinely applicable graph-based retrieval implementation included in the comparison table alongside the three required architectures.